

Sintesis: Parametro Alpha y Comparaciones con Tesis de Nico

(Notebook: combined_findings)

Abstract

Este documento cubre dos cosas: (1) el parametro α que controla como el solver hibrido elige pools, y (2) comparaciones detalladas con resultados de la tesis de Nico sobre testing clasico. Los arboles de decision basicos estan en el documento 01, y el parametro β y la verificacion de Gibbs en el documento 02.

1 El Parametro Alpha (α): Aclarar vs Informar en el Solver Hibrido

1.1 Contexto: que es el solver hibrido?

El solver hibrido hace K pasos con greedy y despues cambia a DP exacto (ver documento 01, seccion 10). Durante los pasos greedy, necesita decidir que pool testear.

1.2 Que es α ?

α controla como el solver hibrido puntua cada pool candidato:

$$\text{score}_\alpha = \alpha \times \underbrace{P(r=0) \cdot \sum u_i}_{\text{aclarar}} + (1 - \alpha) \times \underbrace{\text{ganancia_de_info}}_{\text{aprender}}$$

- $\alpha = 1$: solo le importa aclarar (puntaje miope estandar).
- $\alpha = 0$: solo le importa aprender (maximiza reduccion de incertidumbre).
- $\alpha = 0.5$: mitad y mitad.

Analogia: α es como elegir entre “preguntar algo facil en un examen” ($\alpha = 1$, seguro sacas puntos) vs “preguntar algo que te ayuda a entender el resto del examen” ($\alpha = 0$, inviertes en informacion).

1.3 Diferencia entre α y β

1.4 Nota: hay otro α en `semi_utility`

El archivo `semi_utility.py` define un α diferente:

$$U_{\text{semi}}(\alpha) = \sum u_i [\alpha \cdot P(\text{sano}_i \mid \text{historia}) + (1 - \alpha) \cdot \mathbf{1}_{\text{aclarado}}(i)]$$

Aqui $\alpha = 0$ es utilidad por aclarar (binario: aclarado o no), y $\alpha = 1$ es utilidad por probabilidad de estar sano. **No confundir** con el α del solver hibrido. En este documento nos referimos solo al α del solver hibrido.

	α (documento 03)	β (documento 02)
Donde se usa	Solver hibrido	Greedy puro
Como funciona	Pondera aclarar vs info	Suma bonus de info al score
Formula	$\alpha \cdot \text{aclarar} + (1 - \alpha) \cdot \text{info}$	$\text{aclarar} + \beta \cdot \text{info}$
Rango	$[0, 1]$	$[0, \infty)$
Codigo	<code>hybrid_solver.py</code>	<code>infection_reward_greedy.py</code>

Table 1: α y β son similares pero se usan en contextos distintos.

1.5 Experimento: EU vs α

$n = 6$, $B = 4$, $G = 3$, $p = [0.3]^6$, $u = [1]^6$. Probamos $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$.

Resultado: el mejor α depende de la instancia. Generalmente, un valor intermedio ($\alpha \approx 0.5$) funciona bien. $\alpha = 0$ (solo informacion) y $\alpha = 1$ (solo aclarar) suelen ser peores que valores intermedios.

1.6 Resumen de α

α intermedio (~ 0.5) funciona bien en general. A prevalencia extrema (muy baja o muy alta), no importa mucho. Para detalles de β , ver documento 02.

2 Comparaciones con la Tesis de Nico

La tesis de Nico estudia pooled testing clasico (resultado binario). Nosotros extendemos al caso aumentado (resultado = conteo). Reproducimos sus ejemplos clave para ver que cambia.

2.1 Ejemplo 4.4: Greedy elige el pool equivocado

$n = 3$, $B = 2$, $G = 3$, $p = [1/4, 1/4, 1/4]$, $u = [1, 1, 1]$.

Nico mostro que greedy clasico puede elegir un mal pool de seguimiento despues de un resultado positivo.

En nuestro framework aumentado, el optimo dinamico y el clasico dan la misma EU ($= 1.875$) en este caso — el espacio es tan chico ($n = 3$) que no hay diferencia. Sin embargo, **greedy secuencial** ($= 1.6875$) si pierde calidad, confirmando que la historia completa importa.

Resultado: optimo $>$ greedy; aumentado = clasico en este caso chico.

2.2 Ejemplo 6.2: Trade-off entre EU media y riesgo

$n = 3$, $B = 2$, $G = 3$, $p = [0.9, 0.5, 0.5]$, $u = [1, 1, 1]$.

Nota: p_i es probabilidad de infeccion, asi que persona A ($p = 0.9$) esta *casi seguro infectada*.

En este caso todas las estrategias dan EU = 1.0. El espacio es demasiado chico para que haya diferencias. El punto de Nico (trade-off entre EU media y riesgo de cola) se observa mejor a n mas grande, como en nuestro Part 6.

2.3 Ejemplo 6.3: Efecto de G

$n = 4$, $B = 2$, $p = [1.0, 0.75, 0.75, 0.75]$, $u = [1, 1, 1, 1]$. Persona A esta seguro infectada ($p = 1.0$). Variamos G .

En este caso extremo ($p_0 = 1$), las estrategias coinciden ($EU = 0.5$ para todo G). El conteo aumentado no agrega informacion cuando una persona ya es seguro infectada.

2.4 Escalamiento con B

$n = 5$, $G = 3$, $B = 1, 2, 3, 4$. 30 instancias aleatorias por B .

La ventaja del aumentado sobre el clasico es mayor a B intermedio. A B grande (cercano a n), todos los individuos se resuelven y las estrategias convergen. Este resultado se confirma con mas detalle en el documento principal (paper_findings, Part 3B).

2.5 Ejemplo 6.1: maxima divergencia

$n = 3$, $B = 2$, $G = 3$, $p = [0.91, 0.17, 0.69]$, $u = [0.5, 0.9, 0.33]$.

Nico identifica esta instancia como la de mayor gap dinamico vs estatico. En nuestro framework, todas las estrategias dan EU similar (≈ 0.849) en este caso chico. La divergencia que Nico observa se da en el framework clasico con diferentes restricciones de pool.

2.6 El problema de la “persona segura”

La estrategia dinamica tiende a incluir a la persona menos infectada como ancla en multiples pools. Funciona bien en promedio, pero si esa persona esta infectada, todos los pools que la incluian se invalidan.

Los datos muestran que con $\beta = 0$ o $\beta = 0.5$ el algoritmo ya excluye al ancla en algunos casos. Con $\beta \geq 1$, el algoritmo puede incluirlo mas (no menos), lo cual baja la EU. El efecto de β en este problema es no trivial y depende del caso.

3 Resumen

1. α (solver hibrido): valor intermedio ~ 0.5 funciona bien.
2. En los ejemplos de Nico ($n = 3$), aumentado $\approx$ clasico. El espacio es demasiado chico para que el conteo agregue valor.
3. La ventaja del aumentado se manifiesta mejor a n y B mas grandes (ver paper_findings, Parts 2–4).
4. Greedy secuencial si pierde calidad vs optimo, incluso a $n = 3$.
5. El problema de la “persona segura” existe: el efecto de β es no trivial y depende del caso (ver documento 02 para detalles de β).